BÁO CÁO TRIỂN KHAI

HỆ THỐNG SOAR SỬ DỤNG MCP VÀ AI

Nghiên cứu và triển khai hệ thống SOAR để vá lỗ hổng bảo mật

Nền tảng:	Kali Linux, Docker Compose
AI Engine:	Groq LLaMA 3.3 70B
Transport:	MCP over SSE
SIEM:	Wazuh Indexer
Dashboard:	SOC DSS Platform
Alert:	Telegram Bot

Mục lục

1 Yêu cầu hệ thống **2**

2 Cấu trúc thư mục **2**

3 Cấu hình môi trường **2**
3.1 Tạo Docker network . . . 2
3.2 Clone 2 repo về máy . . . 2
3.3 Tạo file .env cho SOAR . . . 3
3.4 Cấu hình log nguồn (Target App) . . . 3
3.5 Cấu hình Wazuh . . . 3

4 Chạy DSS Platform (Dashboard) **3**

5 Chạy SOAR Pipeline **4**

6 Luồng hoạt động chi tiết **5**
6.1 Trigger 1 — Wazuh Alert (Nhánh A: Auto Execute) . . . 5
6.2 Trigger 2 — Behavioral AI (Nhánh B: Human-in-the-loop) . . . 6
6.3 Trigger 3 — Telegram Callback (Human-in-the-loop) . . . 6
6.4 DSS Integration . . . 7

7 Kiểm tra hoạt động **8**
7.1 Kiểm tra firewall agent . . . 8
7.2 Test block IP thủ công . . . 8
7.3 Test push DSS thủ công . . . 8
7.4 Xem logs real-time . . . 8

8 Mount log hệ thống thật **8**

9 Reset hệ thống **9**

10 Troubleshooting **9**

Yêu cầu hệ thống

Thành phần	Yêu cầu
OS	Kali Linux hoặc Ubuntu 22+
Docker	Docker Engine 24+ và Docker Compose v2
RAM	Tối thiểu 8GB (khuyến nghị 16GB)
Disk	Tối thiểu 20GB trống
Wazuh SIEM	Dang chạy, port 9200 accessible
Target App	Bất kỳ web app nào có Apache/Nginx access.log
Groq API Key	Đăng ký miễn phí tại console.groq.com
Telegram Bot	Tạo bot qua @BotFather

Cấu trúc thư mục

Hai thư mục chính cần thiết:

```
# Repo 1: soar-mcp-platform (nhanh truong-duc-hiep)
mcp-project/
+-- mcp_servers/          # 5 MCP servers
|   +-- collector_server/ # Thu thap log
|   +-- translator_server/# Dich log -> behavior
|   +-- correlator_server/# Tich luy queue + risk score
|   +-- reasoning_server/ # AI phan tich (Groq)
|   +-- response_server/  # Telegram + block IP + DSS
+-- mcp_host/app/         # Orchestrator dieu phoi pipeline
+-- firewall_agent/       # HTTP server chay iptables that
+-- docker-compose.yml
+-- .env                  # API keys (khong push Git)
+-- .env.example          # Mau config
+-- reports/              # Incident reports tu dong (.md)

# Repo 2: soc-dss-platform (nhanh truong-duc-hiep)
soc-dss-platform/
+-- backend/              # Spring Boot API (co AlertClassifier.
   java da sua)
+-- frontend/             # React TypeScript
+-- deploy/production/    # Docker Compose production
```

Cấu hình môi trường

Tạo Docker network

```
docker network create backend_net --subnet 172.30.0.0/24
```

Clone 2 repo về máy

```
# Repo 1: SOAR pipeline
git clone https://github.com/Cygnus-Security/soar-mcp-platform.git
cd soar-mcp-platform
```

```
git checkout truong-duc-hiep
cd week7_24-05-2026/source_code/mcp-project

# Repo 2: DSS Platform
git clone https://github.com/Cygnus-Security/soc-dss-platform.git
cd soc-dss-platform
git checkout truong-duc-hiep
```

Tạo file .env cho SOAR

Copy file mẫu và điền giá trị thật:

```
cd mcp-project
cp .env.example .env
nano .env
```

Nội dung file `.env`:

```
AI_API_KEY=your_groq_api_key          # console.groq.com
TELEGRAM_BOT_TOKEN=your_bot_token     # @BotFather
TELEGRAM_CHAT_ID=your_chat_id         # ID admin nhan alert
DSS_PASSWORD=your_dss_password        # Password DSS
```

Cấu hình log nguồn (Target App)

Mở `docker-compose.yml`, tìm phần `collector_server` và sửa volumes:

```
collector_server:
  volumes:
    - /path/to/your/app/logs:/shared_logs:ro
```

> Hệ thống hỗ trợ bất kỳ web app nào có Apache/Nginx access.log. Không bắt buộc phải dùng DVWA.

Cấu hình Wazuh

Mở `mcp_host/app/config.py` và sửa:

```
WAZUH_INDEXER_URL = "https://YOUR_WAZUH_IP:9200"
WAZUH_USERNAME    = "admin"
WAZUH_PASSWORD    = "SecretPassword"
```

Chạy DSS Platform (Dashboard)

```
# Da clone va checkout truong-duc-hiep o buoc tren
cd soc-dss-platform/deploy/production
cp .env.example .env
# Sua port neu bi chiem: doi 8080 -> 8090 trong docker-compose.yml
docker compose up -d --build
```

Kiểm tra:

```
curl http://localhost:8090/api/v1/health
# Ket qua: {"status":"UP"}
```

Mở dashboard tại `http://localhost:8090` — đăng nhập `admin/admin` (sẽ yêu cầu đổi mật khẩu lần đầu).

> Nếu port 8090 bị chiếm, đổi trong `docker-compose.yml` của DSS:
> `ports: - "8091:80"`

Chạy SOAR Pipeline

```
# Da clone va checkout truong-duc-hiep o buoc tren
cd soar-mcp-platform/week7_24-05-2026/source_code/mcp-project

# Khoi tao data
echo "{}" > data/ip_since.json
mkdir -p reports

# Build va chay
docker compose up -d --build

# Kiem tra containers
docker compose ps
```

Sau khi các container healthy, khởi động SOAR Host:

```
docker exec -it mcp_soar_host python3 -u -m app.main
```

Terminal sẽ hiện:

```
[+] SOAR Host started -- Dual trigger mode
[+] Trigger 1: Wazuh alerts (real-time attack detection)
[+] Trigger 2: Web log polling (behavioral chain monitoring)
[+] Trigger 3: Telegram callback polling (human-in-the-loop)
```

Luồng hoạt động chi tiết

Hệ thống có 3 trigger song song, tất cả đi qua cùng một pipeline MCP-native gồm 5 bước: **Collect → Translate → Correlate → Reason → Respond**.

Sơ đồ tổng quan

```
[Log Sources: web / firewall / db / auth* / audit*]
        |
        v
  Collector MCP -- thu thap raw log, since filter
        |
        v
  Translator MCP -- dich log -> behavioral event (20+ regex)
        |
        v
  Correlator MCP -- tich luy queue, risk score, flush logic
        |
        v
  SOAR Host (orchestrator) <----------- [Wazuh SIEM]
        |
        +-- Trigger 1 (Wazuh alert) -----> Nhanh A: Auto Execute
        |
        +-- Trigger 2 (web log / 15s) ---> Nhanh B: Behavioral AI
        |
        +-- Trigger 3 (Telegram / 5s) ---> Xu ly callback APPROVE/
           REJECT
        |
        v
  Reasoning MCP (Groq AI) -- phan tich behavioral chain
        |
        v
  Response MCP --> Telegram + iptables + Incident Report + DSS

* auth, audit: da code, comment lai, mount sau khi deploy
   production
```

Trigger 1 — Wazuh Alert (Nhánh A: Auto Execute)

Khi Wazuh phát hiện attack đã biết qua signature rules, nó gửi alert đến SOAR. Đây là nhánh ưu tiên cao nhất — **thực thi ngay lập tức**, không chờ admin.

Bước	Hành động	Chi tiết
1	Wazuh gửi alert	Rule level >= 5, có srcip trong data.srcip
2	Fetch 7 nguồn log	web + auth + db + audit + container + firewall + wazuh
3	Translate behaviors	20+ regex pattern, DVWA safe path whitelist
4	Correlate queue	Risk score tích lũy, flush ngay vì has_wazuh_alert=True
5	Execute response ngay	Block IP qua firewall_agent (iptables DROP)

Bước	Hành động	Chi tiết
6	Telegram thông báo	Gửi alert kèm actions executed, không cần admin duyệt
7	AI Incident Report	Groq phân tích full chain, đề xuất patch cụ thể
8	Generate .md report	Lưu /reports/ + gửi Telegram dưới dạng file document
9	Push to DSS	Auto login DSS + push alert NDJSON + trigger correlate

Kết quả: IP attacker bị DROP trong iptables ngay lập tức. Admin nhận Telegram alert kèm file incident report .md. IP chỉ bị block 1 lần trong session — lần sau `already_blocked` → skip.

Trigger 2 — Behavioral AI (Nhánh B: Human-in-the-loop)

Poll web log mỗi 15 giây. Tích lũy behavioral events vào queue. Khi đủ ngưỡng (30 events hoặc risk ≥ 40), gửi toàn bộ chain cho AI phân tích. **Đây là nhánh phát hiện những attack mà Wazuh bỏ sót qua pattern tích lũy.**

Bước	Hành động	Chi tiết
1	Poll web log / 15s	Lấy log mới hơn timestamp cuối (since filter)
2	Fetch 3 nguồn	web + db + firewall (không lấy Wazuh)
3	Translate behaviors	sqli / xss / lfi / cmdi / brute_force / page_browse...
4	Correlate + buffer	Queue tích lũy, risk score có time decay ($e^{-t/300}$)
5	Flush khi đủ ngưỡng	queue ≥ 30 hoặc risk ≥ 40 hoặc idle ≥ 30 phút
6	AI behavioral analysis	$\text{payload}_n = \text{context_memory}(\text{AI}_{n-1}) + \text{behavior_queue}_n$
7a	Nếu benign	Chỉ log forensic, không alert, queue giữ 30 phút
7b	Nếu suspicious	Gửi Telegram với nút APPROVE / REJECT
8a	Admin APPROVE	Block IP iptables + generate report + push DSS
8b	Admin REJECT	Chỉ log, tiếp tục monitor, không block
8c	Timeout 15 phút	Tự động APPROVE nếu không có phản hồi
9	Dedup per session	Mỗi IP chỉ hỏi 1 lần — lần sau already_blocked → skip

Công thức context memory: $\text{payload}_n = \text{context_memory}(\text{AI}_{n-1}) + \text{behavior_queue}_n$ AI luôn nhận kết quả phân tích lần trước + hành vi mới, cho phép phát hiện attack phân tán theo thời gian mà từng event riêng lẻ không đủ để trigger Wazuh.

Trigger 3 — Telegram Callback (Human-in-the-loop)

Poll Telegram `getUpdates` mỗi 5 giây để nhận callback khi admin nhấn APPROVE/REJECT.

- **[APPROVE] Block IP** → gọi firewall_agent → iptables DROP → generate report

- **[REJECT] Chỉ log** → ghi log, tiếp tục monitor, không block
- **[Timeout 15 phút]** → tự động APPROVE nếu không có phản hồi

DSS Integration

Sau mỗi incident (cả nhánh A và B), SOAR tự động push alert lên SOC DSS Platform. DSS **không tự thu thập log** — nó chỉ nhận kết quả đã phân tích từ SOAR.

SOAR		DSS
generate_incident_report()	→	dss_connector.py
Convert sang Wazuh alert format	→	Auto login (session cookie + CSRF)
Push multipart file (NDJSON)	→	POST /api/v1/import/wazuh-alerts
Trigger correlate	→	POST /api/v1/incidents/correlate (async 202)
DSS tính risk score riêng	→	$0.30 \times \text{severity} + 0.20 \times \text{criticality} + \ldots$
Dashboard hiển thị	→	Incidents, Risk Distribution, Recommendations

AlertClassifier DSS: SOAR push alert với `rule.id = SOAR-XSS-*`, `SOAR-SQLI-*`... DSS tự phân loại: XSS Attack, SQL Injection Attack, Brute Force Attack...

Kiểm tra hoạt động

Kiểm tra firewall agent

```
curl http://localhost:9999/health
# Ket qua: {"status":"ok"}
```

Test block IP thủ công

```
docker exec mcp_response_server python3 -c "
import requests
r = requests.post('http://172.17.0.1:9999/block',
    json={'ip': '1.2.3.4', 'duration': 60})
print(r.json())
"
sudo iptables -L INPUT -n | grep 1.2.3.4
# Ket qua: DROP  all  --  1.2.3.4  0.0.0.0/0
```

Test push DSS thủ công

```
docker exec mcp_response_server python3 -c "
import sys; sys.path.insert(0, '/app')
from dss_connector import push_incident_to_dss
r = push_incident_to_dss('10.0.0.1',
    {'severity':'high', 'attack_type':'sqli',
     'risk_summary':'SQL Injection Test',
     'verdict':'attack_confirmed', 'confidence':0.95,
     'session_risk':80.0, 'attack_chain':['sqli'],
     'mode':'incident_report'},
    {'raw_events_collected':10, 'behaviors_translated':10,
     'sources_queried':['web']})
print(r)
"
```

Xem logs real-time

```
docker logs mcp_soar_host -f
docker logs mcp_reasoning_server -f
```

Mount log hệ thống thật

Khi deploy trên app thật, bật thêm các log source bằng cách bỏ comment trong `mcp_host/app/orchestrato`
và mount volume:

```
# docker-compose.yml
collector_server:
  volumes:
    - /var/log/auth.log:/var/log/auth.log:ro
    - /var/log/audit:/var/log/audit:ro
```

```
    - /path/to/nginx/access.log:/shared_logs/access.log:ro
```

> Các log nguồn đang disabled (auth, audit) đã được code đầy đủ trong `collector_server/server.py` — chỉ cần bỏ comment và mount volume.

Reset hệ thống

```
cd mcp-project

# Reset session data
echo "{}" > data/ip_since.json
docker exec mcp_correlator_server rm -f /app/session_store.json

# Restart toan bo
docker compose down
docker compose up -d

# Xoa iptables rules cu
sudo iptables -F INPUT
```

Troubleshooting

Lỗi	Nguyên nhân	Giải pháp
port already allocated	Port bị chiếm	Đổi port trong docker-compose.yml
AI_API_KEY not set	Thiếu file .env	Kiểm tra và tạo file .env
Connection timeout firewall_agent	Container chưa chạy	`docker logs mcp_firewall_agent`
401 DSS	Sai password	Kiểm tra DSS_PASSWORD trong .env
no_events_collected	Sai đường dẫn log	Kiểm tra volumes trong docker-compose.yml
Groq 429 Rate Limit	Quá nhiều request	Chờ 1 phút hoặc đổi API key
analysis_failed	Groq timeout	ai_engine.py đã có retry 3 lần tự động
DSS push 401	Session expired	dss_connector tự login lại tự động
backend_net not found	Network chưa tạo	`docker network create backend_net`
SyntaxError orchestrator	Debug print cũ	Rebuild: `docker compose build soar_host`